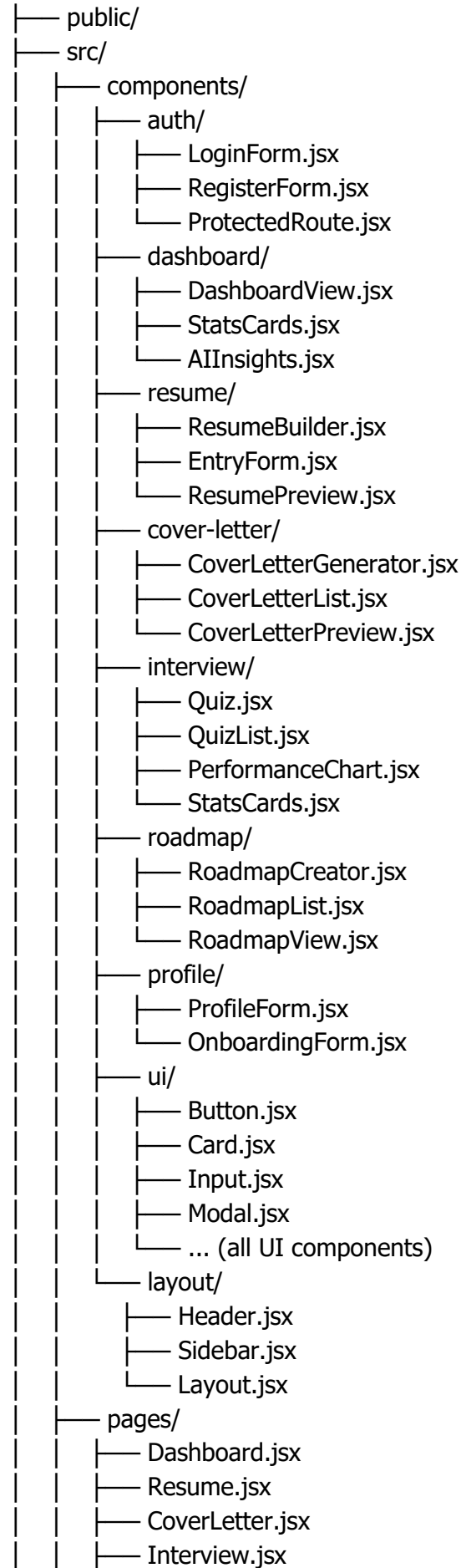
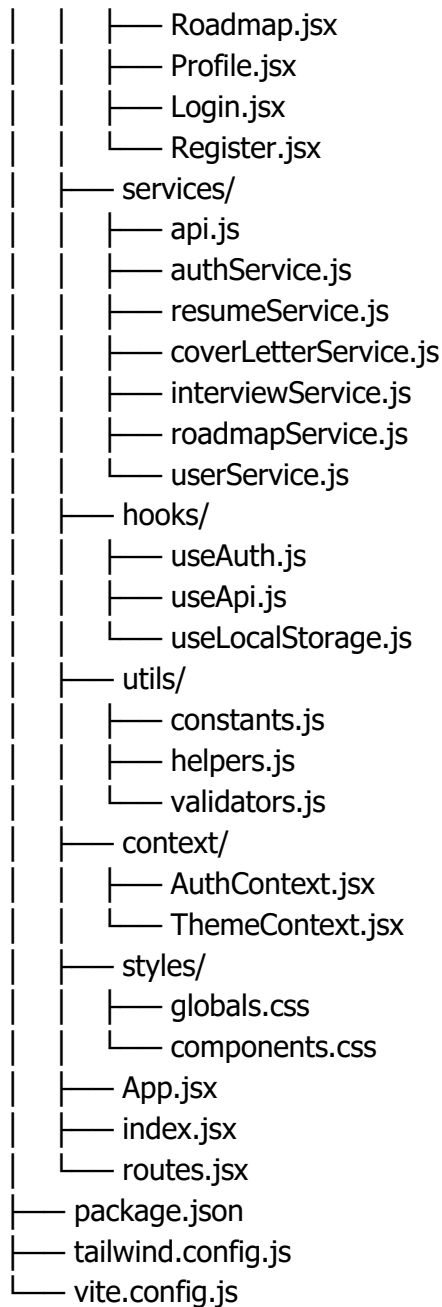


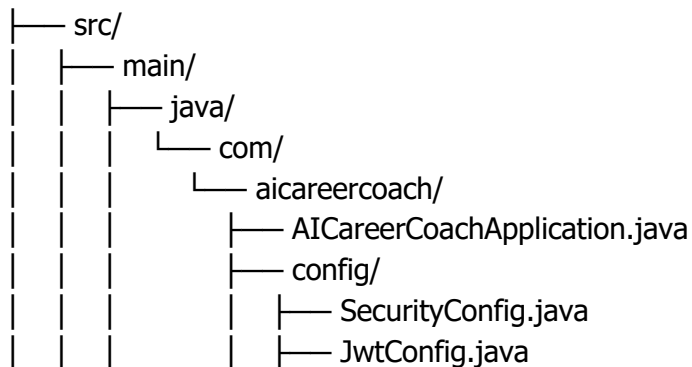
ai-career-coach-frontend/



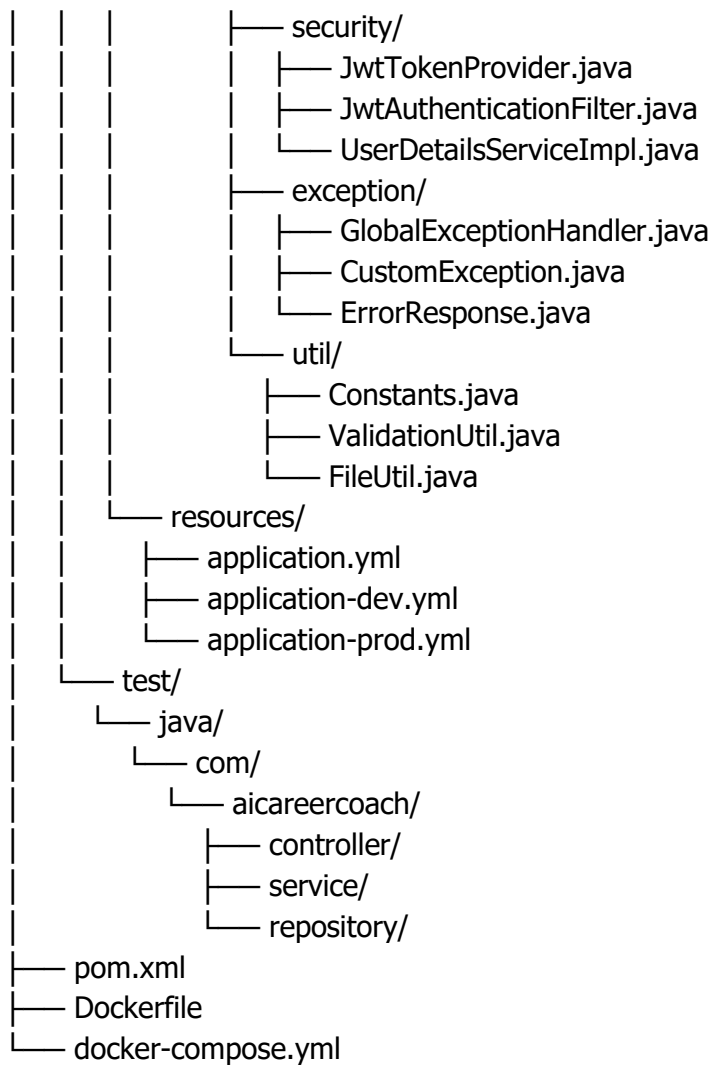


Backend ---

ai-career-coach-backend/



					— CorsConfig.java
					— WebConfig.java
					— MongoConfig.java
					— controller/
					— AuthController.java
					— UserController.java
					— ResumeController.java
					— CoverLetterController.java
					— InterviewController.java
					— RoadmapController.java
					— DashboardController.java
					— IndustryController.java
					— service/
					— AuthService.java
					— UserService.java
					— ResumeService.java
					— CoverLetterService.java
					— InterviewService.java
					— RoadmapService.java
					— DashboardService.java
					— IndustryService.java
					— AIService.java
					— EmailService.java
					— repository/
					— UserRepository.java
					— ResumeRepository.java
					— CoverLetterRepository.java
					— AssessmentRepository.java
					— RoadmapRepository.java
					— IndustryInsightRepository.java
					— model/
					— User.java
					— Resume.java
					— CoverLetter.java
					— Assessment.java
					— Roadmap.java
					— IndustryInsight.java
					— dto/
					— LoginRequest.java
					— RegisterRequest.java
					— UserProfileDto.java
					— ResumeDto.java
					— CoverLetterDto.java
					— QuizDto.java
					— RoadmapDto.java
					— IndustryInsightDto.java



Frontend (Same)

- **React 18** with Vite
- **React Router** for routing
- **Axios** for API calls
- **React Hook Form** for forms
- **Tailwind CSS** for styling
- **Recharts** for charts
- **React Markdown** for markdown rendering
- **React Spinners** for loading states

Backend (Updated)

- **Spring Boot 3.x**
- **Spring Security** with JWT
- **Spring Data MongoDB** with MongoDB
- **MongoDB** database
- **Google Gemini AI** integration

- **Spring Boot Actuator** for monitoring
- **Swagger/OpenAPI** for API documentation

Collections Structure

1. Users Collection

{} json

```
{
  "_id": "ObjectId",
  "email": "user@example.com",
  "password": "hashedPassword",
  "name": "John Doe",
  "imageUrl": "https://example.com/image.jpg",
  "industry": "Technology",
  "bio": "Experienced software developer...",
  "experience": 5,
  "skills": ["JavaScript", "React", "Node.js"],
  "createdAt": "2024-01-01T00:00:00Z",
  "updatedAt": "2024-01-01T00:00:00Z"
}
```

2. Resumes Collection

{} json

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "content": "Resume content in markdown...",
  "atsScore": 85.5,
  "feedback": "Good resume structure...",
  "createdAt": "2024-01-01T00:00:00Z",
  "updatedAt": "2024-01-01T00:00:00Z"
}
```

3. Cover Letters Collection

{} json

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "content": "Cover letter content...",
  "jobDescription": "Job description text...",
  "companyName": "Tech Corp",
  "jobTitle": "Senior Developer",
  "status": "completed",
  "createdAt": "2024-01-01T00:00:00Z",
  "updatedAt": "2024-01-01T00:00:00Z"
}
```

```

{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "quizScore": 85.0,
  "questions": [
    {
      "question": "What is React?",
      "options": ["A library", "A framework", "A language", "A tool"],
      "correctAnswer": "A library",
      "explanation": "React is a JavaScript library..."
    }
  ],
  "category": "Frontend Development",
  "improvementTip": "Focus on state management...",
  "createdAt": "2024-01-01T00:00:00Z",
  "updatedAt": "2024-01-01T00:00:00Z"
}

```

5. Roadmaps Collection

{} json

```

{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "industry": "Technology",
  "role": "Full Stack Developer",
  "description": "Career path description...",
  "skills": ["JavaScript", "React", "Node.js", "MongoDB"],
  "steps": [
    {
      "title": "Learn JavaScript Fundamentals",
      "description": "Master JavaScript basics...",
      "duration": "4 weeks",
      "resources": [
        {
          "type": "course",
          "title": "JavaScript Course",
          "url": "https://example.com/course"
        }
      ]
    }
  ]
}

```

```
{
  "_id": "ObjectId",
  "industry": "Technology",
  "salaryRanges": [
    {
      "role": "Junior Developer",
      "min": 50000,
      "max": 80000,
      "median": 65000,
      "location": "United States"
    }
  ],
  "growthRate": 15.5,
  "demandLevel": "High",
  "topSkills": ["JavaScript", "Python", "React", "Node.js"],
  "marketOutlook": "Positive outlook with strong growth potential",
  "keyTrends": ["AI/ML", "Cloud Computing", "Remote Work"],
  "recommendedSkills": ["TypeScript", "Docker", "AWS"],
  "lastUpdated": "2024-01-01T00:00:00Z",
  "nextUpdate": "2024-01-08T00:00:00Z"
}
```

@ Add Context

In a **Spring Boot project**, a **DTO (Data Transfer Object)** is a plain Java object used to **transfer data between layers** (such as controller ↔ service ↔ repository) without exposing the internal entity models.

◆ Why Use DTO?

1. **✓ Encapsulation** – Hide entity internals and only expose necessary fields.
2. **✓ Security** – Prevent over-posting or data leaks by not exposing sensitive fields.
3. **✓ Custom Response** – Send only the required data in the response, not the whole entity.
4. **✓ Separation of Concerns** – Decouple domain model from API structure.
5. **✓ Improved Performance** – Reduces payload size (important for REST APIs).

Technical Challenges & Solutions

Challenge 1: AI Integration Complexity

"One of the biggest challenges was integrating Google Gemini AI effectively. I solved this by:

- Creating a dedicated AI service layer in Spring Boot
- Implementing proper prompt engineering for consistent outputs
- Adding rate limiting and error handling for API calls
- Structuring responses to handle JSON parsing reliably"

Challenge 2: State Management

"Managing complex state across multiple features was challenging. I addressed this by:

- Using React Context for global state like user authentication
- Implementing custom hooks for API calls and data fetching
- Using React Hook Form for form state management
- Creating reusable components for consistent UI patterns"

Challenge 3: Data Modeling

"Designing the MongoDB schema was crucial. I approached this by:

- Creating flexible document structures for varying resume formats
- Using embedded documents for quiz questions and roadmap steps
- Implementing proper indexing for performance
- Designing for scalability as the user base grows"

How did you handle AI prompt engineering?

Discuss:

- Structured prompts for consistent outputs
- JSON response formatting
- Error handling for AI failures
- Prompt optimization for better results

What indexing did you implement?

Key indexes:

- User email (unique)
- UserId for user-specific queries
- Industry for insights
- CreatedAt for sorting

"How did you integrate Google Gemini AI into your project?"

High-Level Overview

"I integrated Google Gemini AI as the core intelligence engine for generating personalized career content. The AI handles four main features: cover letter generation, interview quiz creation, career roadmap planning, and resume analysis."

Technical Implementation

1. Service Layer Architecture

"I created a dedicated AI service layer in Spring Boot to handle all Gemini interactions. This follows the single responsibility principle and makes the AI integration maintainable and testable."

2. Prompt Engineering Strategy

"The most critical aspect was prompt engineering. I designed structured prompts that consistently produce the exact JSON format I needed."

3. Response Parsing and Error Handling

"I implemented robust response parsing because AI responses can be unpredictable. I clean the response, handle JSON parsing errors, and provide fallback strategies:"

AI Service Integration Points

- **Backend ↔ Google Gemini API:** Direct API calls for content generation
- **Cover Letter Generation:** User profile + job description → AI prompt → formatted cover letter
- **Quiz Generation:** Industry + skills → AI prompt → multiple choice questions
- **Roadmap Generation:** Target role + user background → AI prompt → step-by-step career path
- **Resume Analysis:** Resume content → AI prompt → ATS score + feedback

State Management Architecture

1. Global State with React Context

"For application-wide state like authentication and user profile, I used React Context to avoid prop drilling and ensure consistent access across components. This included user authentication status, profile information, and global settings that needed to be accessible throughout the application."

2. Custom Hooks for API State

"I created custom hooks to manage API calls, loading states, and error handling. This provided a clean abstraction and reusable pattern across the application. Each hook handled the complete lifecycle of an API call - from initial loading to success or error states."

3. Form State Management

"For complex forms like resume builder and cover letter generator, I used React Hook Form. This provided excellent performance, validation, and error handling without unnecessary re-renders. It was especially important for the resume builder which had multiple sections and complex validation rules."

"How did you design the REST API?"

5-Line Answer:

"I designed the REST API following RESTful principles with clear resource-based URLs, proper HTTP methods (GET, POST, PUT, DELETE), and consistent status codes. I implemented a layered architecture with controllers handling HTTP requests, services containing business logic, and repositories managing data access."

"How did you handle error handling?"

Backend Error Handling

"I created a global exception handler using @ControllerAdvice that caught all exceptions and returned consistent error responses. This included validation errors (400), authentication failures (401), resource not found (404), and server errors (500). Each error response included a status code, message, timestamp, and optional error details for debugging."

Custom Exception Classes

"I created specific exception classes for different scenarios like UserNotFoundException, AIServiceException, and ValidationException. This allowed for precise error handling and meaningful error messages based on the specific error type."

Frontend Error Handling

"I implemented React error boundaries to catch component errors and prevent the entire application from crashing."

1. User Authentication & Registration

Implementation Approach

"I implemented a complete authentication system using Spring Security with JWT tokens. The registration process collected user information including name, email, password, and industry, then created a user profile with hashed passwords using bcrypt. The login system validated credentials and generated JWT tokens with user claims for subsequent API access."

Key Components

"The system included email validation, password strength requirements, industry selection, and automatic profile creation. I implemented proper error handling for duplicate emails and invalid credentials, with user-friendly error messages and secure token management."

2. Resume Builder with ATS Scoring

Implementation Approach

"I created a comprehensive resume builder that allowed users to input their information in markdown format. The system integrated with Google Gemini AI to analyze resumes for ATS optimization, providing scores and specific feedback on structure, keywords, and formatting."

Key Components

"The resume builder included sections for personal information, experience, education, skills, and achievements. The AI analysis checked for keyword optimization, proper formatting, and industry-specific requirements. Users could export resumes as PDFs and track multiple versions."

3. AI-Powered Cover Letter Generator

Implementation Approach

"I implemented an AI cover letter generator that analyzed job descriptions and user profiles to create personalized cover letters. The system used Google Gemini AI with carefully crafted prompts that included user skills, experience, and the specific job requirements."

Key Components

"The system required job title, company name, and job description as input. The AI generated professional cover letters highlighting relevant skills and experience, maintaining proper business letter formatting and keeping content concise and impactful."

4. Interview Quiz System

Implementation Approach

"I created a dynamic quiz system that generated industry-specific interview questions using AI. The system created multiple-choice questions based on the user's industry and skills, with explanations for correct answers and improvement tips."

Key Components

"The quiz system generated 10 questions per session with 4 options each. Questions were tailored to the user's industry and experience level. The system tracked scores, provided immediate feedback, and stored assessment history for progress tracking."

5. Career Roadmap Generator

Implementation Approach

"I implemented an AI-powered career roadmap generator that created personalized learning paths based on target roles and user background. The system analyzed industry requirements and user skills to generate step-by-step career development plans."

Key Components

"The roadmap included detailed steps with timeframes, learning resources, and skill requirements. Each step had specific actions, estimated duration, and recommended resources like courses, books, and projects. The system tracked progress and provided motivation."

6. Industry Insights & Trends

Implementation Approach

"I implemented an automated system that generated industry insights using AI analysis. The system provided salary ranges, growth rates, demand levels, and recommended skills for different industries and roles."

Key Components

"The system included salary data for different experience levels, growth projections, top skills in demand, and market outlook. It provided personalized recommendations based on user industry and experience level."

Why gemini ai

- **Superior text generation quality** for professional content
- **Excellent JSON response formatting** for structured data
- **Advanced reasoning capabilities** for complex prompts

- **Reliable API with good documentation**
- **Consistent output quality** with minimal post-processing
- **Better cost-effectiveness** compared to other AI models
- **Stable and predictable performance** for production use

For Career Coaching Applications

- **GPT-4**: Best quality, highest cost
- **Claude 3 Opus**: Excellent reasoning, high cost
- **Gemini Pro**: Good balance, cost-effective
- **GPT-3.5**: Fast, affordable, limited reasoning

How did you implement the PDF download feature?"

5-Line Answer:

"I implemented PDF generation using server-side processing with libraries like iText or Apache PDFBox in Spring Boot. The system converted markdown content to HTML using a markdown parser, then generated PDFs with professional styling and formatting. I created secure download endpoints that generated PDFs on-demand

Security Measures ==

Authentication & Authorization

"Forms the foundation of security - without proper authentication, all other measures are ineffective."

HTTPS & CORS

"Protects data in transit and prevents cross-origin attacks that could compromise user data."

Input Validation

"Prevents the most common attack vectors - injection attacks and XSS that can lead to data breaches."

Rate Limiting

"Protects against abuse and ensures service availability while preventing brute force attacks."

Data Encryption

"Ensures data confidentiality even if other security measures fail."

Error Handling

"Prevents information disclosure that could help attackers understand system vulnerabilities."

File Upload Security

"Critical for preventing malware and unauthorized access through file uploads."

AI Service Security

"Protects against prompt injection and ensures secure integration with external AI services."

Session Security

"Protects user sessions and ensures proper data handling compliance."

Monitoring

"Enables early detection and response to security incidents before they become breaches."

1. Axios

Q: What is Axios and why do we use it in React?

Answer:

Axios is a promise-based HTTP client used to make requests from the browser to APIs. In React, it is commonly used to fetch or send data to REST APIs. It supports features like interceptors, request/response transformation, and error handling, making it more powerful than the native `fetch()` API.

Key Features:

- Automatically transforms JSON data.
- Supports request cancellation.
- Easy to configure headers (e.g., for authentication).
- Works in both client and Node.js environments.

2. Recharts

Q: What is Recharts and where have you used it?

Answer:

Recharts is a React charting library that helps in visualizing data using components like bar charts, pie charts, and line graphs. It is built on D3.js but provides a more declarative and easy-to-use API for React.

Why I used it:

I used Recharts in a dashboard to visualize user activity and performance metrics. It allows easy integration of dynamic data and is highly customizable.

Example Components:

- `<LineChart>`, `<BarChart>`, `<PieChart>`, `<Tooltip>`, `<XAxis>`, `<YAxis>`
-

3. React Markdown

Q: What is React Markdown and how have you used it?

Answer:

React Markdown is a React component that safely renders Markdown-formatted text as HTML. It's commonly used in note-taking apps, blogs, or any application that supports user-generated content in Markdown format.

Use Case:

In my notes-sharing platform, I used React Markdown to render notes written in Markdown syntax into properly formatted HTML content.

Security Note:

It avoids using `dangerouslySetInnerHTML`, making it safer for rendering untrusted content.

4. Spring Security

Q: What is Spring Security and how have you implemented it?

Answer:

Spring Security is a powerful authentication and authorization framework for Spring-based applications. It helps secure REST APIs or web apps by managing user roles, passwords, sessions, CSRF protection, etc.

Key Concepts:

- **Authentication:** Verifying user identity.
- **Authorization:** Controlling access based on roles/permissions.
- **JWT:** Commonly used for stateless authentication.

- **Custom Filters:** For login, token validation, etc.

Usage Example:

I used Spring Security to secure backend APIs with JWT, implementing filters for login and role-based access.

✓ 5. Spring Data MongoDB

Q: What is Spring Data MongoDB?**Answer:**

Spring Data MongoDB is a part of the Spring Data project that provides seamless integration between Spring Boot and MongoDB. It abstracts MongoDB operations into simple repository interfaces.

Features:

- CRUD operations with `MongoRepository`
 - Automatic query generation
 - Custom queries using `@Query`
 - Works well with documents, embedded documents, and collections
-

✓ 6. Swagger

Q: What is Swagger and why is it important?**Answer:**

Swagger (now known as OpenAPI) is a toolset for documenting and testing RESTful APIs. It generates an interactive UI where developers can explore and test endpoints without writing any client code.

Usage in Spring Boot:

- I used Swagger (via `springdoc-openapi`) to automatically document APIs and provide a UI at `/swagger-ui.html`.

Benefits:

- Enhances API visibility for frontend/backend teams
- Useful for external developers consuming your API
- Supports live testing of endpoints with headers, params, and body